

CSCE 678 DISTRIBUTED SYSTEMS AND CLOUD COMPUTING · MODULE 01 · LECTURE NOTES

Partial Failure

The thing that makes all of this hard.

MODULE OUTCOMES

Explain partial failure and why it has no single-machine analogue.

Explain why a timeout cannot distinguish the cases it must.

State the eight fallacies and give a consequence of each.

Distinguish the failure models and say which to assume.

Decide honestly whether to distribute at all.

1 The central difficulty

ON ONE MACHINE, EVERYTHING FAILS TOGETHER

A function call either returns a result, or the process dies and the question of what it returned is moot. There is no third outcome, and that fact is quietly load-bearing in every single-machine program ever written.

So single-machine error handling has two cases, and one of them — the process dying — you do not have to handle, because there is nobody left to handle it.

In a distributed system, components fail independently, and the components still running must carry on *without knowing what happened to the others*. Some of them are fine, some are gone, some are merely unreachable, and from the inside these look identical.

That is partial failure, and it has no single-machine analogue. It is worth being precise about this: distribution is not a harder version of a problem you already know how to solve. **It introduces a failure mode that simply does not exist on one machine,** and essentially everything in this course is a response to it.

You send a request. The timeout expires. What happened?

- (a) the request was lost on the way out
-> the server never saw it. Safe to retry.
- (b) the server executed it and the REPLY was lost
-> already done. Retrying does it TWICE.
- (c) the server is still working on it
-> retrying may run it CONCURRENTLY with itself.
- (d) the server is dead
-> retry elsewhere, if there is an elsewhere.
- (e) the network is partitioned; the server is fine and is serving other clients perfectly well
-> YOU are the one cut off. Retrying achieves nothing until the partition heals.

THESE ARE INDISTINGUISHABLE FROM THE CALLER. No measurement, no protocol and no amount of waiting separates them, because the evidence is on the far side of the thing that is broken.

A longer timeout only makes you slower to be wrong.

Every difficulty in this course descends from that ambiguity. Idempotence, deduplication, consensus, linearizability, leases, fencing tokens, and client-side prediction are all responses to it, and recognising them as such is most of what makes the subject coherent rather than a list of techniques.

THE RESPONSES, AND WHAT EACH ACTUALLY BUYS

Make operations idempotent. If applying an operation twice has the same effect as applying it once, then cases (a) and (b) stop differing and the caller can simply retry. **This is the cheapest fix available and it should be the default** — design the operation to be idempotent rather than designing the protocol to avoid duplicates.

Assign a unique request identifier and deduplicate at the server. This is idempotence bolted onto operations that are not naturally idempotent (incrementing a counter, appending to a list), at the cost of the server remembering which identifiers it has seen — which is itself state that must be bounded and replicated.

Use consensus to agree on what happened (Module 06). Correct, expensive, and it does not remove the ambiguity so much as move it into a subsystem built specifically to survive it.

Or accept the ambiguity and reconcile afterwards — which is what eventual consistency (Module 04) and netcode reconciliation (Module 10) both do, in very different settings. **And note: 'exactly once' message delivery does not exist.** It is provably impossible over an unreliable network. **What people mean by it is at-least-once delivery plus idempotent processing**, which produces exactly-once *effects* — a genuinely different and achievable thing, and worth insisting on the distinction when someone's design document claims the former.

2 The eight fallacies

The assumption	The reality	The consequence
The network is reliable.	Packets are lost, duplicated, and reordered as a matter of routine, not as an exception.	Every remote call needs a failure path, and that path needs testing as much as the success path.
Latency is zero.	Physics puts a floor of roughly 100 ms on a round trip across the world, and real routes are worse.	Chatty interfaces collapse. See the callout below.
Bandwidth is infinite.	It is shared, variable, and metered.	Payload size is a design decision, not an implementation detail.
The network is secure.	It is not, and the inside of your datacentre is not either.	Authenticate every hop and assume hostility — which in a game means assuming the client is the attacker (Module 11).
Topology does not change.	Nodes come and go constantly; addresses change under you.	Service discovery, not configuration files.
There is one administrator.	There are many, and they are not coordinated.	Version skew is permanent, so every protocol change must be compatible in both directions for a while.
Transport cost is zero.	Serialisation costs CPU and egress costs money.	Cloud bills are substantially egress charges, which surprises people who budgeted for compute (Module 12).
The network is homogeneous.	Nothing is.	Protocols must negotiate rather than assume.

LATENCY IS A PHYSICS PROBLEM AND IT DOES NOT IMPROVE

Light travels through optical fibre at roughly 200,000 km per second — about two-thirds of its speed in vacuum, because of the refractive index of glass. New York to London and back is about 56 ms at the theoretical best, and real fibre routes are not great circles.

Bandwidth has improved by orders of magnitude over four decades. Latency has not, and cannot. It is bounded below by the speed of light and the physical route, and the remaining overhead is switching and queueing, which are already small.

So a design that makes ten sequential round trips costs ten times the RTT, regardless of how fast the link is. Upgrading the connection does nothing; the requests are waiting on each other, not on the wire.

Batch requests, pipeline them, and move computation to the data. This is precisely CSCE 735 Module 10's $\alpha + n\beta$ model — the latency term is paid per message regardless of size, so fewer and larger is better. **The same conclusion, derived twice in two different courses**, which suggests it is a property of communication rather than of any particular system.

3 Failure models

Model	Nodes may	What it costs to tolerate
Crash-stop	Stop, and never return.	The simplest model to reason about, and often unrealistic — real machines reboot and come back.
Crash-recovery	Stop and later return, possibly having lost everything not written durably.	Requires durable state and careful recovery, because a returning node must not contradict what it promised before it crashed. The right model for most internal systems.
Omission	Drop messages, sent or received.	Mostly covered by retries and idempotence (§1).
Byzantine	Behave arbitrarily, including lying, equivocating, and colluding.	Requires $3f+1$ nodes to tolerate f failures rather than $2f+1$, with substantially more messages and cryptographic signing. Expensive, and sometimes unavoidable.
Fail-stop	Stop, and the failure is reliably detectable by others.	Extremely convenient and almost never actually available — §1 explains why detection is the hard part.

Assume crash-recovery for systems you control, and Byzantine wherever a participant may be an adversary. That second case is not exotic: **in a multiplayer game, every client is a potentially Byzantine node** running on hardware the attacker owns, with a debugger attached (Module 11). The game industry's answer — a single authoritative server that trusts no client — is a pragmatic alternative to Byzantine consensus that works because there is a natural trusted party.

4 Whether to distribute at all

DISTRIBUTION IS A COST PAID FOR A SPECIFIC BENEFIT

The legitimate reasons are three. The data or the load genuinely exceeds what one machine can hold or handle; the users are geographically distributed and latency to a single location would be unacceptable; or **the system must survive the failure of a machine**, which no single machine can do by definition.

The illegitimate ones are also three. It seems more professional; microservices are what everyone does; the team is large so the architecture should be too (Conway's law applied backwards, as a justification rather than a warning).

What it costs: partial failure in every code path, no global clock (Module 03), debugging that spans processes and machines, consistency decisions that must be made explicitly (Module 04), and **an operational burden that never goes away** (Module 13).

A single machine with two terabytes of memory and 128 cores handles a great deal. This is the third time this program has reached that conclusion — CSCE 608 Module 13 argued it about databases and CSCE 735 Module 10 argued it about MPI — and three independent derivations of the same advice is reasonable evidence that the reflex to distribute is both strong and frequently unexamined.

- **Go or Rust**, with a trivial TCP and UDP echo client and server working. Everything after Module 02 builds on this.
- **A network impairment tool** — tc netem on Linux, Clumsy on Windows, or a small proxy of your own that delays and drops. **Add 200 ms of latency and 5% loss to your echo and watch what happens**; it is the single most useful ten minutes in the course.
- **Wireshark**, and a capture of your own traffic. **Looking at the actual bytes once** makes Module 02 concrete in a way no description does.
- **A plotting setup that shows percentiles rather than means**. Module 13 explains at length why means are useless here, and it is easier to have built the right thing from the start.
- **Docker Compose**, so that running five nodes on one machine is one command. **A system you can only run as a single instance will not get tested as a distributed one**, and localhost without impairment lies about everything that matters.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 6.5840 — lecture 1, introduction (free)	The §1 framing and the course's overall shape. Watch it this week.
Deutsch & Gosling — The Eight Fallacies of Distributed Computing (free)	The §2 list, from the people who wrote it down. One page, and still entirely current.
Peter Bailis & Kyle Kingsbury — The Network Is Reliable (free)	Evidence for the first fallacy , collected from real production incidents. Read it before deciding your network is fine.
Kleppmann — Designing Data-Intensive Applications, chapter 8 ('The Trouble with Distributed Systems')	The best single treatment of §1 and §3 , including unreliable clocks and the process-pause problem.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write a TCP and a UDP echo client and server.
- 2 **Add 200 ms latency and 5% loss** with an impairment tool and measure both. Report the difference in behaviour.
- 3 **Construct each of the five timeout cases** from §1 deliberately, and confirm your client cannot tell them apart.

- 4 Implement a non-idempotent operation (increment a counter) and demonstrate a double-apply under retry.
- 5 **Make it idempotent with a request ID and deduplication**, and show the double-apply is gone.
- 6 Measure RTT to five servers around the world and compare against the speed-of-light floor for each distance.
- 7 **Build an interface that makes ten sequential round trips**, then batch it into one. Measure both at 5 ms and at 200 ms RTT.
- 8 Capture your echo traffic in Wireshark and identify the handshake, the data, and the teardown.
- 9 Take a system you have built and **write down honestly whether it needs to be distributed**, against §4's three legitimate reasons.
- 10 Set up Docker Compose running five instances of your echo server.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What is partial failure, and why does it have no single-machine analogue?
- 2 Give the five things a timeout cannot distinguish, and say why a longer timeout does not help.
- 3 Name four responses to the ambiguity and what each buys.
- 4 Why does 'exactly once' delivery not exist, and what do people mean by it?
- 5 State the eight fallacies with a consequence of each.
- 6 Why has latency not improved the way bandwidth has?
- 7 Name five failure models and say which to assume when.
- 8 Why is every game client a potentially Byzantine node?
- 9 Give three legitimate and three illegitimate reasons to distribute.